Variables et affectation

Variables

L'informatique désigne le traitement automatique de l'information[1]. Dans les programmes informatiques, l'information est représentée par des données.

Cours

Les programmes informatiques manipulent des données en utilisant des **variables**. Une variable permet d'associer un nom à une valeur.

Une variable informatique peut se concevoir comme une sorte de “boite” étiquetée avec un nom, dans laquelle un programme enregistre une valeur pour la consulter ou la modifier pendant son exécution[2].

En Python, comme dans la plupart des langages informatiques, le nom d'une variable :

- s'écrit en lettres minuscules (« `a` » à « `z` ») et majuscules (« `A` » à « `Z` ») et peut contenir des chiffres (« `0` » à « `9` ») et le caractère blanc souligné (« `_` ») ;
- ne doit pas comporter d'espace, de signes d'opération « `+` », « `-` », « `*` » ou « `/` », ni de caractères spéciaux comme des signes de ponctuation « `'` », « `"` », « `,` », « `.` », « `:` », « `@` », etc. ;
- ne doit pas commencer par un chiffre ;
- ne doit pas être un mot réservé de Python, par exemple « `for` », « `if` », « `print` », etc. ; et
- est sensible à la casse, ce qui signifie que les variables « `TesT` », « `test` » ou « `TEST` » sont différentes.

En pratique cela permet d'éviter les noms de variable réduits à une lettre et d'utiliser des noms qui ont un sens !

PEP 8

La PEP 8 [3] donne un grand nombre de recommandations de style pour écrire du code Python agréable à lire et recommande en particulier de nommer les variables par des mots en minuscule séparés par des blancs soulignés (`_`) [4], par exemple d'appeler une variable `somme_des_nombres` plutôt que `s` dans un programme qui additionne des nombres.

Types de variable

Cours

Les variables peuvent être de **types** différents en fonction des données qu'elles représentent.

Les principaux types de variable sont :

- les nombres entiers (type `int`) ;
- les nombres décimaux, appelés « flottants » (type `float`) qui s'écrivent toujours avec un point (⚠️ le séparateur décimal est un point, **pas une virgule**), par exemple `5.0`.

  Noter que `.5` et `5.` permettent d'écrire rapidement les flottants `0.5` et `5.0` et que `2e5` ou `2E5` (pour 2×10^5) permettent d'écrire le nombre flottant `200000.0` ;
- les booléens prenant seulement les valeurs `True` ou `False` (type `bool`) ;
- les textes ou chaines des caractères (type `str`) écrits entre une paire de guillemets (`"`) ou d'apostrophes (`'`) ;
- d'autres types dits "construits" comme les p_uplets, tableaux, dictionnaires[5], etc.

Affectation

Cours

L'**affectation** consiste à donner une valeur à une variable. En Python, comme dans la plupart des langages informatiques, l'**affectation d'une valeur à une variable** est représentée par le signe « `=` ».[6]

Par exemple, saisir les commandes suivantes dans la console Python permet d'affecter les valeurs 3 (type `int`), 3.0 (type `float`) et "3" (type `str`) à des variables nommées respectivement `a`, `b` et `c` :

La console Python, ou interpréteur Python, est un moyen rapide d'exécuter des commandes. Il suffit de taper une instruction en réponse à l'invite `>>>` puis d'appuyer sur la touche « Entrée » pour lancer son exécution.

PEP 8

Mettre des espaces autour d'un égal (`=`).

```
>>> a = 3
>>> b = 3.0
>>> c = "3"
```

En Python, c'est l'affectation qui définit le type d'une variable[7].

⚠️ C'est bien la valeur qui se trouve à droite du signe « `=` » qui est affectée à la variable à gauche, et pas dans l'autre sens.

```
>>> 3 = a
  File "<interactive input>", line 1
```

```
SyntaxError: can't assign to literal
```

PEP 8

Mettre un espace après une virgule (`,`) , mais pas avant.

Il est aussi possible d'affecter des valeurs à plusieurs variables en même temps :

```
>>> a, b = 3, 4
>>> a
3
>>> b
4
```

et d'affecter la valeur d'une variable à une autre variable, par exemple :

Quand on tape le nom d'une variable dans la console, elle affiche sa valeur.

```
>>> a = 3
>>> b = a
>>> b
3
```

⚠️ Il n'est pas possible d'utiliser une variable avant de lui avoir affecté une valeur.

```
>>> d
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
NameError: name 'd' is not defined
```

Exercice corrigé

On affecte les valeurs 5 et 6 (de type `int`) à deux variables nommées respectivement `a` et `b` :

```
>>> a = 5
>>> b = 6
```

Compléter ces instructions pour échanger les valeurs de `a` et de `b` (sans utiliser les chiffres 5, 6 ou tout autre chiffre).

Réponse

---

1. Le terme « informatique » résulte de l'association du terme « information » au suffixe « -ique » signifiant « qui est propre à ». ↩
2. La notion de variable en informatique diffère des mathématiques. En mathématique une variable apparait dans l'expression symbolique d'une fonction , ou dans une équation pour désigner une

inconnue qu'il faut trouver, ou encore dans une formule comme ^{2} ^{2} ^{2} pour indiquer que l'égalité est vraie pour toutes les valeurs de et . ↩

3. Une PEP (pour *Python Enhancement Proposal*) est un document fournissant des informations à la communauté Python, ou décrivant une nouvelle fonctionnalité. En particulier la PEP 8 décrit les conventions de style de code agréable à lire. ↩

4. Le style qui consiste à nommer les variables par des mots écritsen minuscule séparés par des blancs soulignés, par exemple `somme_des_nombres`, est appelé « *snake case* » en opposition au style qui consiste à écrire les mots attachés en commençant par des majuscules, par exemple `SommeDesNombres`, appelé « *camel case* ». ↩

5. Les p_uplet, tableaux, dictionnaires sont étudiés dans un autre chapitre du programme de 1ère. ↩

6. En algorithmique, l'affectation est symbolisée par une flèche allant de la valeur (à droite) vers le nom de la variable (à gauche), par exemple pour affecter la valeur à la variable . ↩

7. Python est un langage de typage dynamique, ce n'est pas le cas de nombreux langages comme le C ou le C++ qui forcent à définir le type d'une variable et à le conserver au cours de la vie de la variable, ils sont de typage statique. Exemple d'affectation en C :

```
int a;
a = 3;
```

↩